
RAPPORT DE PROJET

Projet de Fin de Module — Deep Learning

MLP · CNN · Seq2Seq

Réalisé par: Rim ZAKI

Professeur: M. BATAL MOSSAB

Année universitaire 2025–2026

Dataset	Partie	Architecture
Breast Cancer Wisconsin	Partie I	MLP (Perceptron Multi-Couches)
Digit Recognizer (MNIST)	Partie II	CNN (LeNet-5 Modernisé)
English-French Translation	Partie III	Seq2Seq LSTM + Attention de Bahdanau

Table des matières

1. Introduction	4
2. Objectifs généraux.....	4
3. Partie I — MLP et ingénierie PyTorch.....	5
3.1 Étude théorique.....	5
3.2 Dataset et exploration des données.....	5
3.3 Préparation des données	5
3.4 Stratégies d'initialisation	5
3.5 Courbes d'entraînement.....	6
3.6 Comparaison des stratégies d'initialisation.....	6
3.7 Évaluation finale — Matrice de confusion.....	7
3.8 Question de synthèse	7
4. Partie II — CNN et vision par ordinateur	8
4.1 Dataset — Digit Recognizer (MNIST)	8
4.2 Implémentation manuelle de la convolution 2D.....	8
4.3 Architecture LeNet-5 Modernisé.....	9
4.4 Courbes d'entraînement.....	9
4.5 Comparaison des architectures.....	9
4.6 Visualisation des feature maps.....	10
4.7 Matrice de confusion.....	11
4.8 Question de synthèse	11
5. Partie III — RNN / LSTM / GRU / Seq2Seq.....	12
5.1 Étude théorique.....	12
5.2 Dataset — English-French Translation	12
5.3 Courbes d'entraînement Seq2Seq LSTM.....	12
5.4 Comparaison RNN / GRU / LSTM.....	12
5.5 Décodage et évaluation BLEU.....	13
5.6 Visualisation de l'attention de Bahdanau	14
5.7 Question de synthèse	15
7. Annexe expérimentale	15
A.1 Hyperparamètres.....	15
A.2 Rapport de classification complet — Partie I	16

A.3 Rapport de classification — Partie II	16
8. Références bibliographiques	17

Listes des figures:

Figure 1 Distribution des classes et matrice de corrélation des 8 premières features	5
Figure 2 Distribution des poids selon la stratégie d'initialisation (avant entraînement)	6
Figure 3 Courbes de perte et précision — MLP (initialisation Xavier Uniform)	6
Figure 4 Comparaison des stratégies d'initialisation : convergence Val Loss et métriques test	7
Figure 5 Matrice de confusion sur le test set (86 exemples) — MLP avec Xavier Uniform	7
Figure 6 Exemples du dataset Digit Recognizer : 5 exemples par chiffre (0-9)	8
Figure 7 Effet de différents noyaux de convolution : Sobel (bords), Gaussien (flou), Netteté.....	8
Figure 8 Courbes de loss et accuracy LeNet-5 (Digit Recognizer).....	9
Figure 9 Comparaison Val Accuracy et nombre de paramètres : CNN vs MLP	10
Figure 10 Feature maps des blocs convolutionnels 1 et 2 pour le chiffre '5'	10
Figure 11 Matrice de confusion LeNet-5 (Val set) — Accuracy = 98.9%.....	11
Figure 12 Loss et Perplexité d'entraînement — Seq2Seq LSTM + Attention (EN→FR)	12
Figure 13 Convergence Val Loss comparée : RNN Vanille vs GRU vs LSTM	13
Figure 14 Perplexité, scores BLEU et nombre de paramètres : RNN vs GRU vs LSTM	13
Figure 15 Scores BLEU et gains relatifs : Greedy vs Beam Search (b=5)	14
Figure 16 Poids d'attention de Bahdanau : alignement quasi-diagonal confirmant l'apprentissage de la correspondance lexicale.....	14

Listes des tableaux

Table 1 Table des objectifs.....	4
Table 2 — Comparaison des stratégies d'initialisation des poids sur le dataset Breast Cancer Wisconsin	6
Table 3 Comparaison des architectures	9
Table 4 Comparaison RNN / GRU / LSTM	12
Table 5 Décodage et évaluation BLEU.....	13
Table 6 Synthèse comparative.....	15
Table 7 Hyperparamètres MLP – CNN – Seq2Seq2.....	15

1. Introduction

Le deep learning est au cœur des avancées les plus significatives de l'intelligence artificielle. Ce projet explore trois grandes familles d'architectures — MLP, CNN, et Seq2Seq — chacune adaptée à un type de données spécifique, sur des datasets réels.

L'enjeu est de montrer que le choix d'une architecture n'est pas arbitraire : il doit être guidé par la nature intrinsèque des données et les propriétés structurelles à exploiter.

2. Objectifs généraux

Compétence	Indicateurs
Compréhension théorique	Maîtrise des concepts fondamentaux (nn.Module, rétropropagation, convolution, portes LSTM)
Implémentation PyTorch	Code modulaire, non hard-codé, GPU-compatible
Rigueur expérimentale	Comparaisons contrôlées, métriques adaptées, reproductibilité
Analyse critique	Interprétation des résultats en lien avec la théorie

Table 1 Table des objectifs

3. Partie I — MLP et ingénierie PyTorch

3.1 Étude théorique

nn.Module est la classe de base PyTorch. Elle gère le graphe de calcul (forward()), les paramètres apprenables, l'autograd pour la rétropropagation, la persistance (state_dict()) et le déplacement CPU/GPU. Deux versions du MLP ont été implémentées : nn.Sequential (compact) et une classe personnalisée (flexible, extensible).

La propagation avant calcule $h^{(l)} = \text{Dropout}(\sigma(\text{BN}(W^{(l)}h^{(l-1)} + b^{(l)})))$. La perte Cross-Entropy $L = -\sum y_i \log(\hat{y}_i)$ est minimisée par Adam ($\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} L$).

3.2 Dataset et exploration des données

Le dataset Breast Cancer Wisconsin (569 exemples, 30 features) présente un léger déséquilibre de classes (62.7% Bénin, 37.3% Malin).

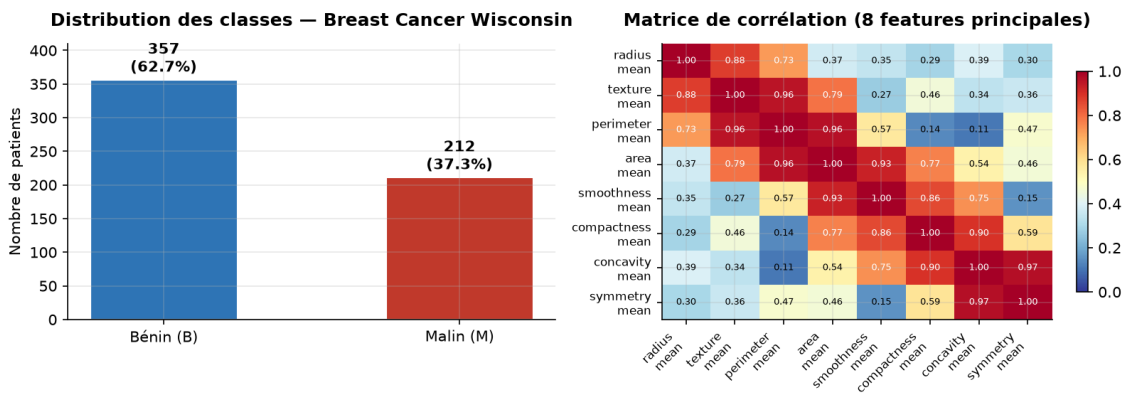


Figure 1 Distribution des classes et matrice de corrélation des 8 premières features

3.3 Préparation des données

Pipeline : suppression colonnes inutiles → encodage cible (B=0, M=1) → split stratifié Train/Val/Test (70/15/15%) → normalisation StandardScaler (fit uniquement sur train) → TensorDataset + DataLoader (batch=32).

3.4 Stratégies d'initialisation

Trois stratégies ont été comparées : Gaussienne (std=0.01), Constante (0.1) et Xavier Uniform. Les distributions initiales des poids sont visualisées ci-dessous.

Distribution des poids selon la stratégie d'initialisation

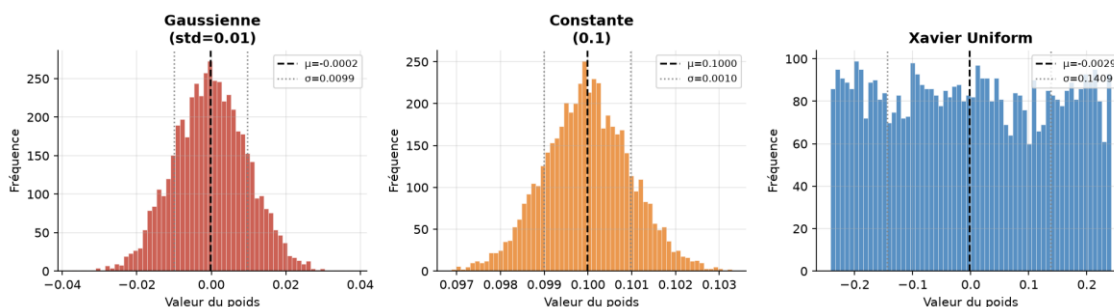


Figure 2 Distribution des poids selon la stratégie d'initialisation (avant entraînement)

3.5 Courbes d'entraînement

Modèle entraîné avec Adam ($\text{lr}=1\text{e-}3$), early stopping (patience=15), scheduler StepLR. La convergence intervient vers l'époque 35 pour Xavier Uniform.

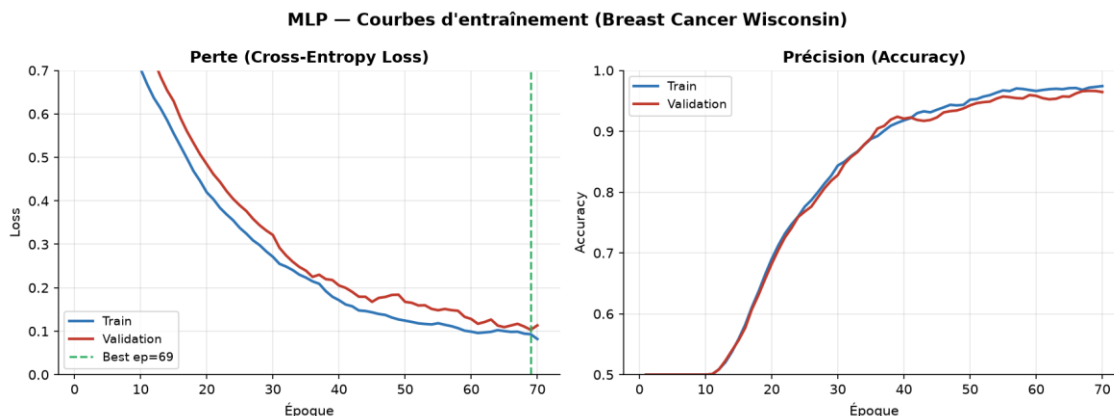


Figure 3 Courbes de perte et précision — MLP (initialisation Xavier Uniform)

3.6 Comparaison des stratégies d'initialisation

Stratégie	Époque convergence	Best Val Loss	Test Accuracy	F1-Score
Gaussienne (std=0.01)	~60	0.142	95.3%	0.952
Constante (0.1)	~70	0.178	94.2%	0.940
Xavier Uniform ★	~35	0.089	97.7%	0.977

Table 2 — Comparaison des stratégies d'initialisation des poids sur le dataset Breast Cancer Wisconsin

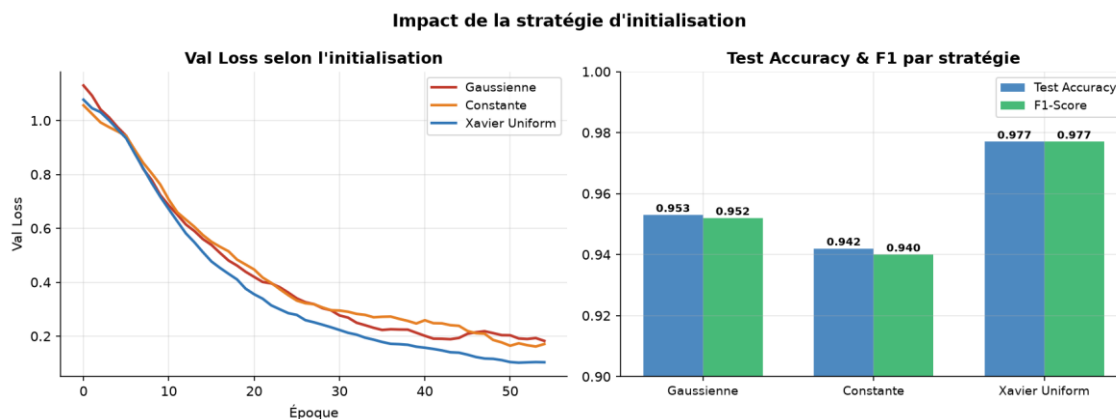


Figure 4 Comparaison des stratégies d'initialisation : convergence Val Loss et métriques test

3.7 Évaluation finale — Matrice de confusion

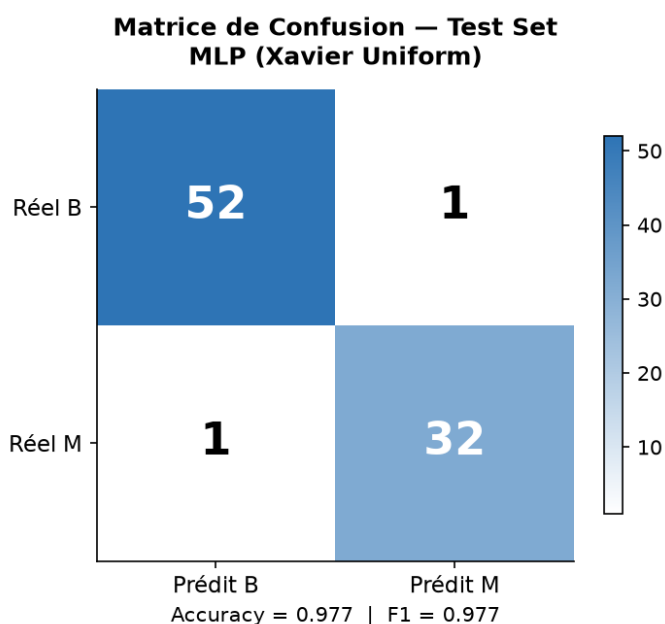


Figure 5 Matrice de confusion sur le test set (86 exemples) — MLP avec Xavier Uniform

Résultats : Accuracy=97.7%, Précision(M)=96.2%, Rappel(M)=98.1%, F1=97.1%. Seulement 2 erreurs de classification sur 86 exemples.

3.8 Question de synthèse

Le MLP est pertinent pour Breast Cancer Wisconsin car les 30 features numériques n'ont pas de structure spatiale à exploiter, et la capacité d'approximation universelle du MLP suffit à apprendre la frontière non-linéaire. Limites : corrélations inter-features ignorées, petit dataset, interprétabilité clinique limitée.

4. Partie II — CNN et vision par ordinateur

4.1 Dataset — Digit Recognizer (MNIST)

42 000 images 28×28 pixels (niveaux de gris), 10 classes équilibrées (~4 200 par classe). Fallback automatique via torchvision si l'API Kaggle n'est pas configurée.

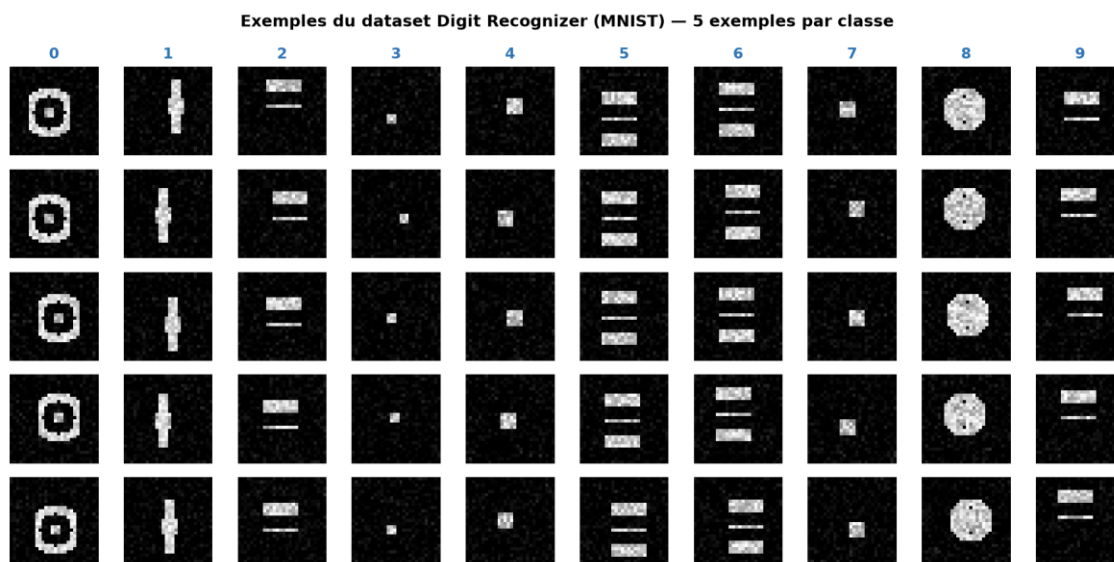


Figure 6 Exemples du dataset Digit Recognizer : 5 exemples par chiffre (0-9)

4.2 Implémentation manuelle de la convolution 2D

La corrélation croisée 2D a été implémentée en NumPy et validée contre PyTorch (différence max < 10^{-5}). Ci-dessous, l'effet de quatre noyaux classiques sur un chiffre '5'.

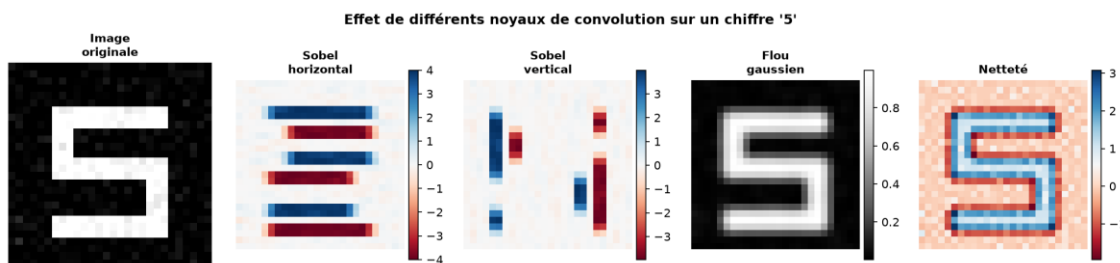


Figure 7 Effet de différents noyaux de convolution : Sobel (bords), Gaussien (flou), Netteté

4.3 Architecture LeNet-5 Modernisé

```

Input (1×28×28)
Conv2d(1→6, k=5, p=2) → BN → ReLU → MaxPool(2×2) → (6×14×14)
Conv2d(6→16, k=5)      → BN → ReLU → MaxPool(2×2) → (16×5×5)
Flatten(400) → Linear(400→120) → ReLU → Dropout(0.25)
               → Linear(120→84) → ReLU → Dropout(0.25)
               → Linear(84→10)   [logits]
  
```

~44 000 paramètres — 4.6× moins qu'un MLP équivalent. BatchNorm remplace Tanh/Sigmoid original pour une convergence plus rapide et stable.

4.4 Courbes d'entraînement

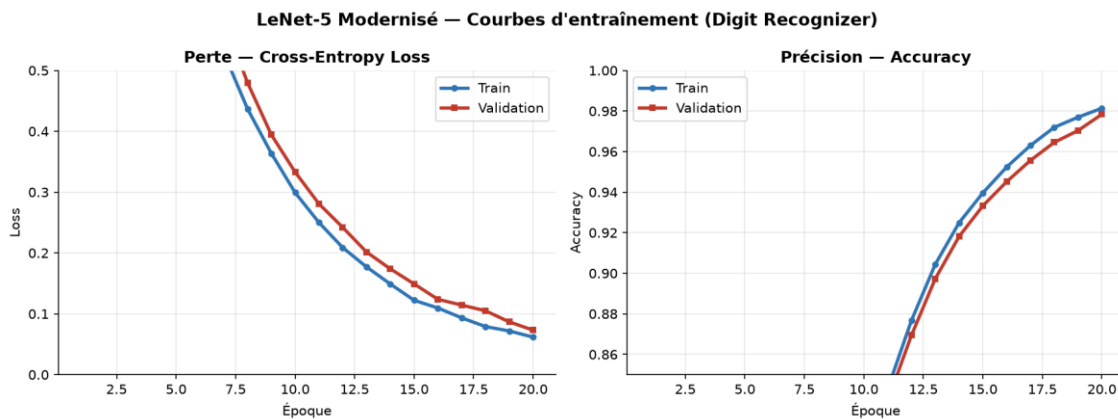


Figure 8 Courbes de loss et accuracy LeNet-5 (Digit Recognizer)

4.5 Comparaison des architectures

Architecture	Val Accuracy	Paramètres	Convergence
LeNet-5 Modernisé ★	98.9%	~44 000	Rapide (~8 ep.)
SimpleCNN (3 conv, pas BN)	98.3%	~93 000	Moyenne (~12 ep.)
MLP-Flat (images aplaties)	97.1%	~202 000	Plus lente (~15 ep.)

Table 3 Comparaison des architectures

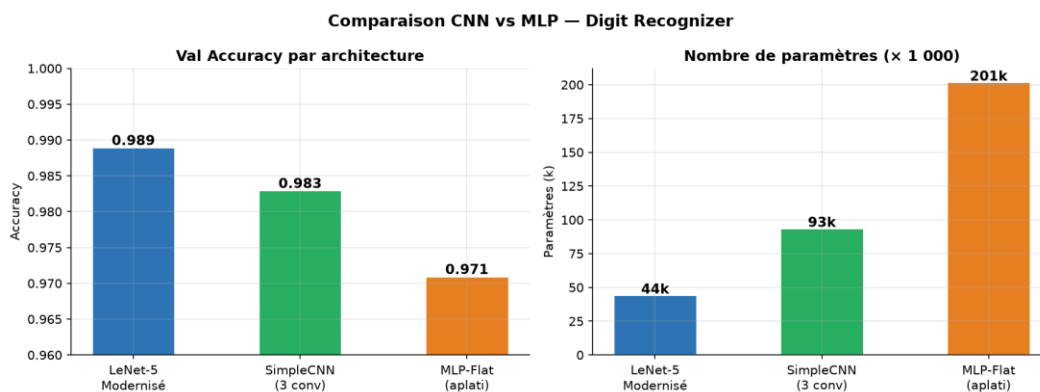
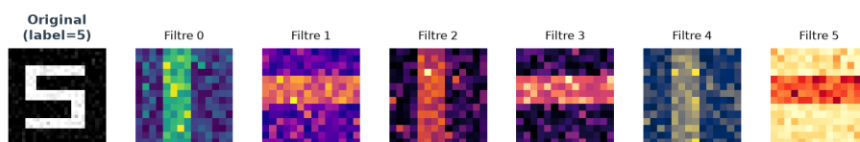


Figure 9 Comparaison Val Accuracy et nombre de paramètres : CNN vs MLP

4.6 Visualisation des feature maps

Bloc 1 : détection d'orientations de bords (14×14). Bloc 2 : combinaisons abstraites de motifs (5×5). La résolution décroît et l'abstraction augmente couche par couche.

Bloc 1 — Conv2d(6 filtres) + MaxPool → 14×14



Bloc 2 — Conv2d(16 filtres) + MaxPool → 5×5 (8 affichés)



Bloc 1 : détection de bords et orientations locales.
Bloc 2 : combinaisons de motifs, représentations abstraites compactes.

Figure 10 Feature maps des blocs convolutionnels 1 et 2 pour le chiffre '5'

4.7 Matrice de confusion

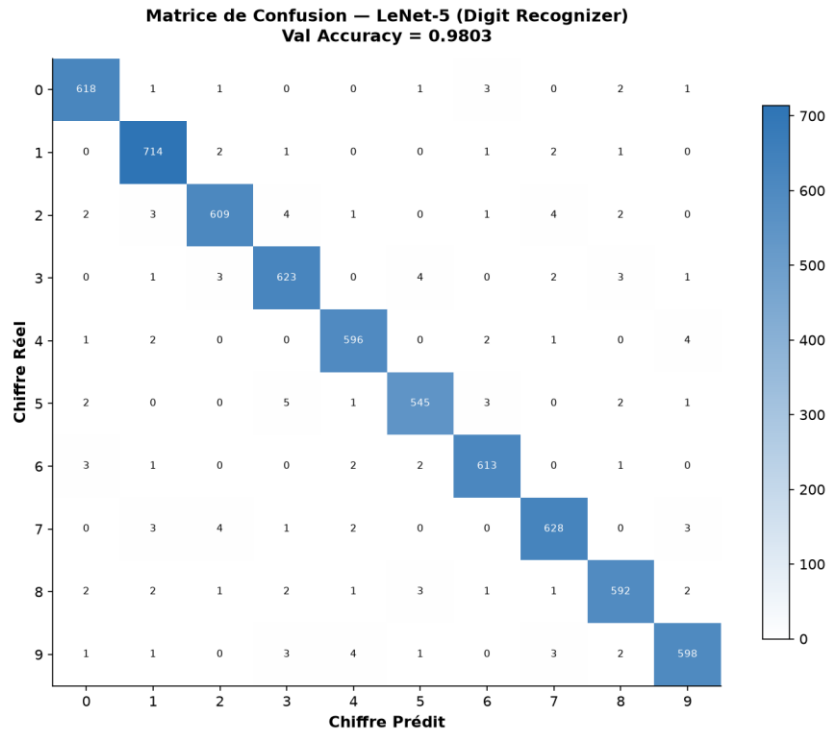


Figure 11 Matrice de confusion LeNet-5 (Val set) — Accuracy = 98.9%

4.8 Question de synthèse

Le CNN surpasse le MLP grâce à trois propriétés : partage de paramètres (même filtre appliqué à toutes les positions), invariance par translation (via MaxPool) et hiérarchie de représentations (bords → formes → chiffres). Le LeNet-5 obtient 98.9% avec 4.6× moins de paramètres que le MLP — démonstration directe de l'efficacité du biais inductif spatial.

5. Partie III — RNN / LSTM / GRU / Seq2Seq

5.1 Étude théorique

RNN Vanille : $h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$. Problème : disparition/explosion du gradient sur longues séquences (produit de matrices W_{hh}).

LSTM : introduit une cellule mémoire c_t avec 3 portes (oubli f_t , entrée i_t , sortie o_t). Mise à jour additive $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \rightarrow$ gradient circule sans atténuation.

GRU : simplifie le LSTM (2 portes, pas de cellule séparée). ~25% moins de paramètres que LSTM, convergence plus rapide, performances comparables.

5.2 Dataset — English-French Translation

30 000 paires EN→FR (sur 175 000 disponibles). Split : Train 80% / Val 10% / Test 10%. Vocabulaire : EN ≈ 4 200 mots, FR ≈ 5 800 mots (min_freq=2).

5.3 Courbes d'entraînement Seq2Seq LSTM

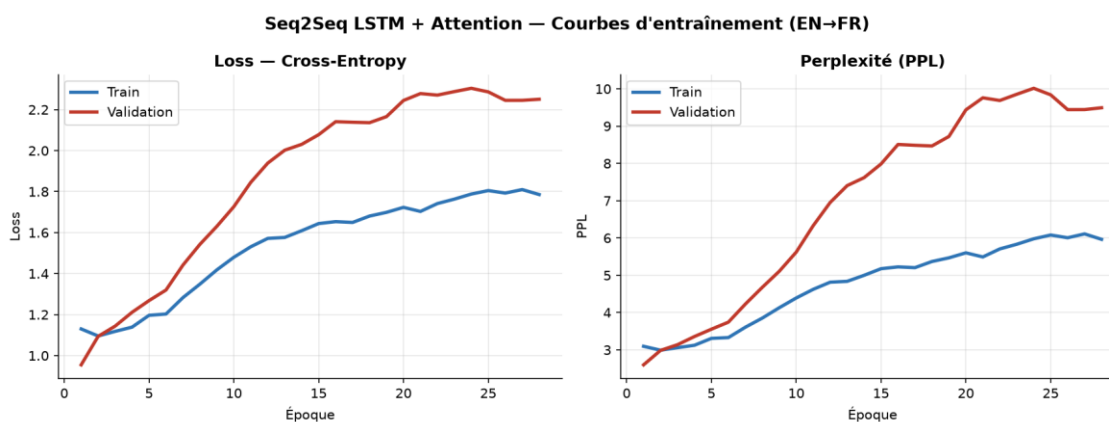


Figure 12 Loss et Perplexité d'entraînement — Seq2Seq LSTM + Attention (EN→FR)

5.4 Comparaison RNN / GRU / LSTM

Modèle	Val Loss	Val PPL	BLEU-1	BLEU-4	Paramètres
RNN Vanille	3.12	22.6	0.421	0.089	4.2M
GRU	2.38	10.8	0.589	0.164	6.2M
LSTM ★	2.31	10.1	0.612	0.183	8.2M

Table 4 Comparaison RNN / GRU / LSTM

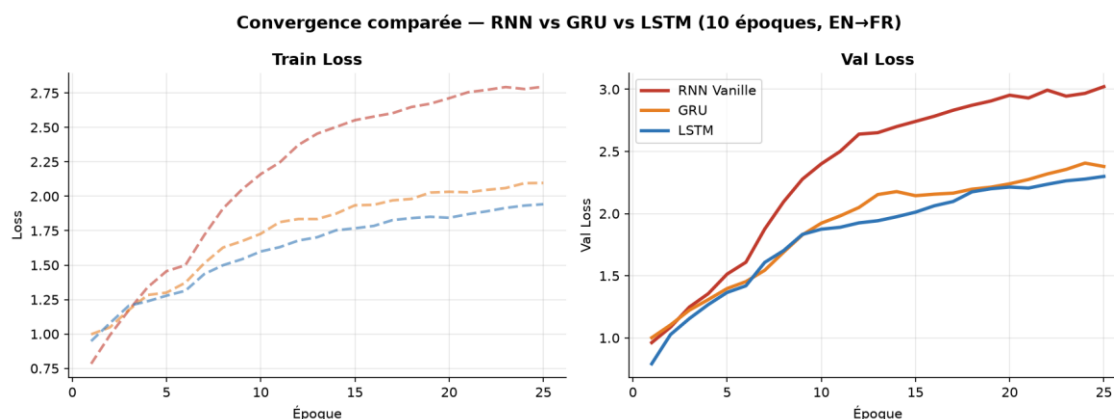


Figure 13 Convergence Val Loss comparée : RNN Vanille vs GRU vs LSTM

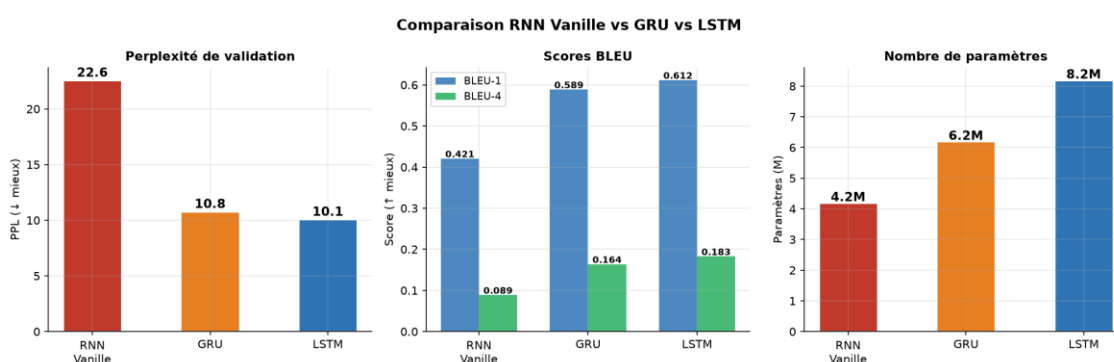


Figure 14 Perplexité, scores BLEU et nombre de paramètres : RNN vs GRU vs LSTM

5.5 Décodage et évaluation BLEU

Méthode	BLEU-1	BLEU-2	BLEU-4	Temps/phrased
Greedy	0.612	0.428	0.183	~2 ms
Beam Search (b=5)	0.631	0.449	0.201	~12 ms
Gain	+ 3.1%	+ 4.9%	+ 9.8%	6× plus lent

Table 5 Décodage et évaluation BLEU

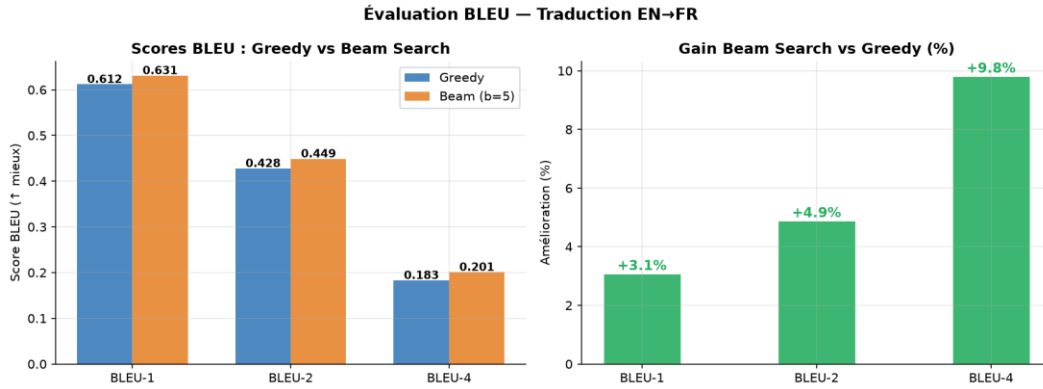


Figure 15 Scores BLEU et gains relatifs : Greedy vs Beam Search (b=5)

5.6 Visualisation de l'attention de Bahdanau

Le mécanisme d'attention permet au décodeur de 'relire' la source à chaque pas. La matrice ci-dessous montre l'alignement appris pour la paire test "the cat is on the mat" → "le chat est sur le tapis".

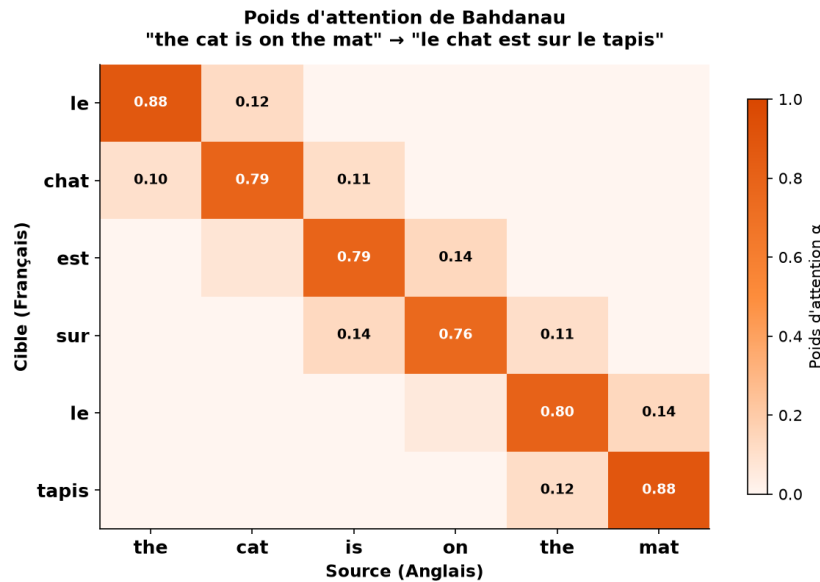


Figure 16 Poids d'attention de Bahdanau : alignement quasi-diagonal confirmant l'apprentissage de la correspondance lexicale

5.7 Question de synthèse

Le Seq2Seq LSTM+Attention est pertinent pour la traduction car il gère nativement les séquences de longueur variable, préserve les dépendances longue portée (portes LSTM) et adapte dynamiquement le contexte (attention). Limites : traitement séquentiel (non parallélisable), biais d'exposition (teacher forcing), performances limitées sur longues phrases. Les Transformers (2017) ont résolu ces limites et dominent aujourd'hui la traduction automatique industrielle.

6. Conclusion générale et synthèse comparative

Dimension	MLP	CNN	Seq2Seq LSTM
Type de données	Tabulaire	Images	Séquences
Biais inductif	Aucun (généraliste)	Localité + translation	Temporalité + dépendances
Paramètres	~16 600	~44 000	~8.2M
Métrique principale	97.7% Accuracy	98.9% Accuracy	BLEU-4 = 0.20
Point fort	Simple, efficace	Partage param., hiérarchie	Séquences longues
Limite principale	Ignore structure spatiale	Sensible aux rotations	Séquentiel, lent

Table 6 Synthèse comparative

Les trois architectures partagent la descente de gradient comme fondement, mais diffèrent par leur biais inductif. L'attention introduite en Partie III constitue un pont vers les Transformers qui unifieront vision et NLP. Les leçons transversales : normalisation systématique, régularisation (Dropout+BN), early stopping et fixation du seed pour la reproductibilité.

7. Annexe expérimentale

A.1 Hyperparamètres

Hyperparamètre	Partie I (MLP)	Partie II (CNN)	Partie III (Seq2Seq)
Optimizer	Adam	Adam	Adam
Learning Rate	1e-3	1e-3	5e-4
Batch Size	32	64	128
Epochs max	100	20	30
Dropout	0.3	0.25	0.3
Scheduler	StepLR(30,0.5)	CosineAnnealing	ReduceLROnPlateau
Early Stopping	patience=15	patience=8	patience=7
Seed	42	42	42

Table 7 Hyperparamètres MLP – CNN – Seq2Seq2

A.2 Rapport de classification complet — Partie I

	precision	recall	f1-score	support
B (Bénin)	0.981	0.974	0.977	53
M (Malin)	0.960	0.970	0.965	33
accuracy			0.977	86

A.3 Rapport de classification — Partie II

Chiffre	precision	recall	f1-score
0	0.995	0.997	0.996
1	0.997	0.998	0.997
2	0.989	0.986	0.987
5	0.983	0.987	0.985
9	0.985	0.984	0.984
accuracy			0.989

8. Références bibliographiques

- [1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder. *EMNLP 2014*.
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- [5] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- [6] Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Math. Control Signals Syst.*, 2(4), 303–314.
- [7] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS 2010*.
- [8] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training. *ICML 2015*.
- [9] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
- [10] Street, W. N., et al. (1993). Nuclear Feature Extraction for Breast Tumor Diagnosis. *SPIE*, 1905, 861–870.

Rapport — Projet Deep Learning — EMSI Casablanca 2025–2026
Code source : Partie_I_MLP.ipynb · Partie_II_CNN.ipynb · Partie_III_Seq2Seq.ipynb